

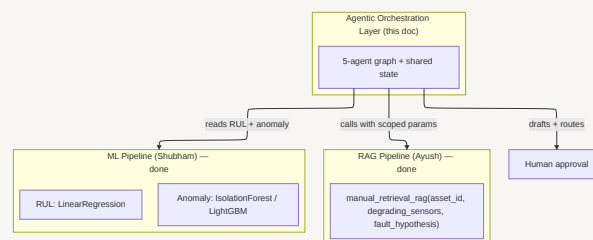


Mech Sage — Agentic Orchestration Layer: Architecture Design



What this layer is. The orchestration layer sits *above* the finished ML pipeline (Shubham) and RAG pipeline (Ayush). It does not predict and it does not retrieve. Its job is to **coordinate**: pull the ML signals, turn them into a structured fault hypothesis, call the RAG tool with the right scope, handle abstention, and drive the flow all the way to a human-approved **work order**.

1. Where this layer sits



Golden rule: the ML models own the *numbers* (RUL, anomaly). The RAG owns the *knowledge* (fault meaning, procedure). The LLM/agents only **reason and phrase** — they never invent an RUL, a failure mode, or a manual fact.

2. Design principles

1. **ML first, RAG second.** Never call RAG before you have the ML signals — the ML output *is* the query.
2. **Structured hypothesis, not free text.** The layer converts ML output into a typed block (`asset_id` , `degrading_sensors` , `fault_hypothesis`) before touching RAG.
3. **Abstention is a first-class path.** `NO_RELEVANT_PASSAGE` / `CORPUS_UNAVAILABLE` are expected outcomes, not crashes — they route to a human, never to a guessed answer.
4. **Human gate before anything irreversible.** No schedule/shutdown is committed without approval.
5. **Everything is traceable.** Every work order carries the RUL, the sensors, the fault, and the manual citation that produced it.

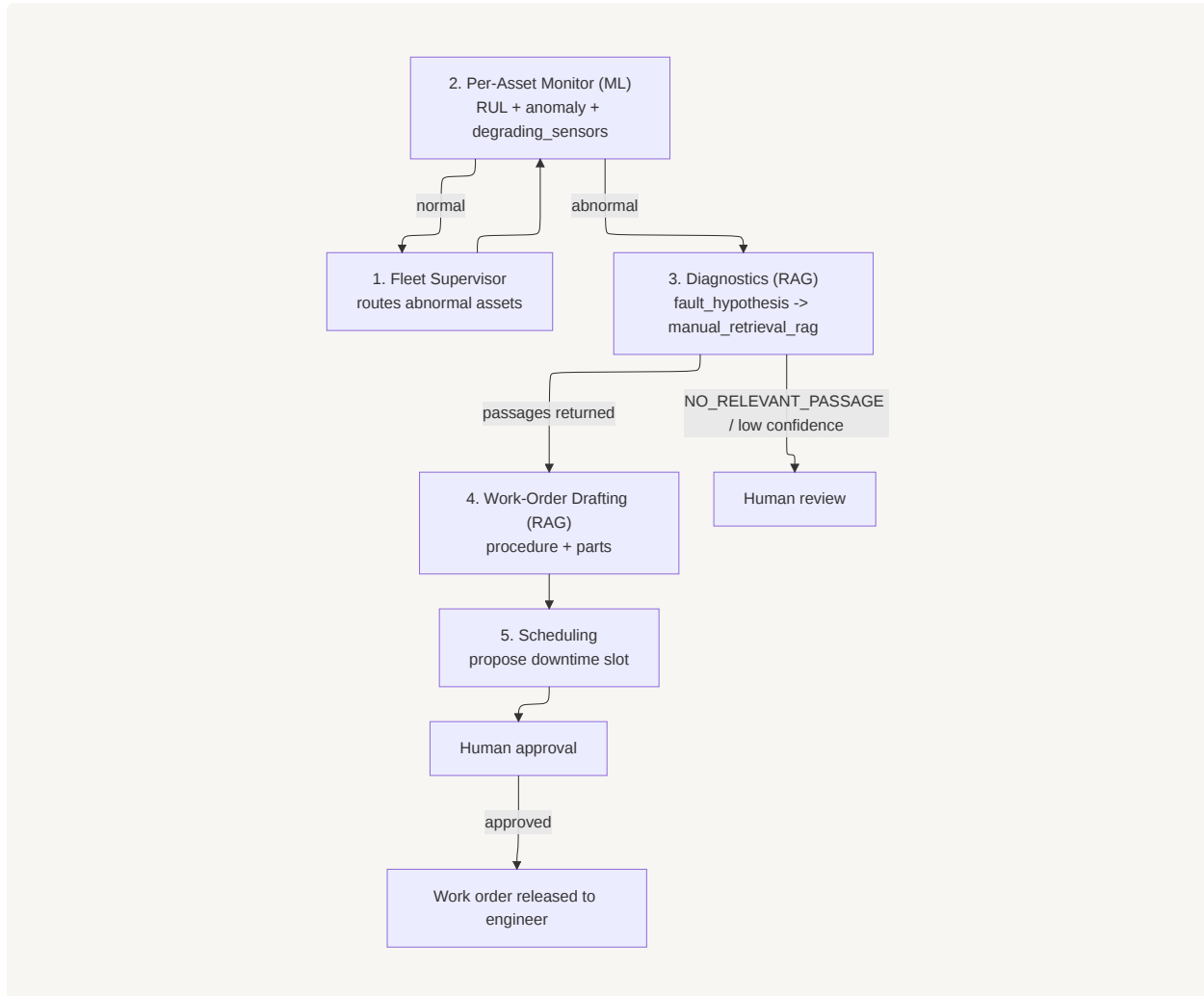
3. The shared state (the contract every agent reads/writes)

This is the single object that flows through the graph. Each agent adds its slice.

Field	Written by	Meaning
<code>asset_id</code>	Supervisor	Routes RAG scope: <code>turbofan</code> / <code>milling_machine</code> / <code>ironside</code>
<code>raw_telemetry</code>	Supervisor	The sensor window for this unit
<code>rul_estimate</code>	Monitor (ML)	Cycles left (from LinearRegression)
<code>anomaly_flag</code> / <code>anomaly_score</code>	Monitor (ML)	From IsolationForest (FD001/003) or LightGBM (FD002/004)
<code>degrading_sensors</code>	Monitor (ML)	Which sensors are moving — derived from SHAP top features + deltas
<code>fault_hypothesis</code>	Diagnostics	Candidate failure mode, from the signature map
<code>retrieved_passages</code>	Diagnostics (RAG)	Top-K scoped passages, or an abstention code
<code>diagnosis</code> • <code>confidence</code> • <code>citation</code>	Diagnostics	Grounded conclusion + manual reference

Field	Written by	Meaning
<code>work_order</code>	Work-Order agent	Structured, plant-language fix instruction
<code>schedule_proposal</code> / <code>approval_status</code>	Scheduling + Human	Proposed downtime slot and its approval

4. The agent graph (5 nodes, hierarchical)



1. Fleet Supervisor

- Watches the fleet, decides which asset needs attention, sets `asset_id` (and thus RAG scope). Cheap model, no tools. Routing only.

2. Per-Asset Monitor — ML boundary

- Calls the pre-trained models: `rul_estimate` (LinearRegression) + `anomaly_flag` (IsolationForest / LightGBM by dataset).
- **Builds** `degrading_sensors` from SHAP top contributors (e.g. `sensor_15`, `sensor_7`) + current sensor deltas. *This is the critical glue step — it turns model internals into the RAG filter.*
- If nothing abnormal → return to Supervisor. No RAG call.

3. Diagnostics — RAG boundary

- Turns `degrading_sensors` into a `fault_hypothesis` using the fault-signature map (e.g. T30↑ + P30↓ → HPC degradation).
- Calls `manual_retrieval_rag(asset_id, degrading_sensors, fault_hypothesis)`.
- Handles the guardrail: if it gets `NO_RELEVANT_PASSAGE` / `CORPUS_UNAVAILABLE` (similarity < 0.42) → **abstain** → **human**, do not proceed.
- Owns the **confidence gate**: low confidence also routes to human.

4. Work-Order Drafting — RAG boundary

- Uses the returned parent passages (procedure + parts) to draft the structured work order in **plain plant language**.
- Generation via the RAG's Gemini generator under RAGAS faithfulness — stays grounded in retrieved passages only.

5. Scheduling → Human approval

- Proposes a downtime slot (plan-and-execute), then **stops at a human gate**. Nothing commits without approval.

5. The glue that is actually *your* work



The two pipelines are done. The new logic you own is the **translation between them**: turning ML output into the exact `{asset_id, degrading_sensors, fault_hypothesis}` block the RAG tool expects, and turning abstention codes into safe routing. Everything else is wiring.

5a. Building `degrading_sensors` : rank sensors by SHAP contribution + recent delta magnitude; take the top movers with direction (up/down). Output shape like

```
["sensor_15↑", "sensor_7↑", "sensor_11↓"] .
```

5b. Building `fault_hypothesis` : map the degrading-sensor signature → candidate failure mode via a small lookup (compressor / turbine / fan / general wear). This is a hint for retrieval, not the final answer — RAG + confidence gate confirm it.

5c. Handling abstention: wrap the RAG call; on `NO_RELEVANT_PASSAGE` / `CORPUS_UNAVAILABLE` , set `diagnosis = "unresolved"` and route to human with the raw ML signal attached. Never call the work-order agent on an empty context.

6. End-to-end trace (one asset)

1. Supervisor flags **Unit 17**, sets `asset_id = "turbofan"` .
2. Monitor: RUL ≈ 22 cycles, anomaly = true, `degrading_sensors = ["sensor_15↑", "sensor_7↑"]` .
3. Diagnostics: hypothesis = HPC degradation → `manual_retrieval_rag("turbofan", [...], "HPC degradation")` → passages returned, confidence 0.87, cite Manual §4.2.
4. Work-Order: drafts WO-2041 (inspect/clean HPC, bearing kit, ~2–3 hrs, LOTO).
5. Scheduling: proposes next maintenance window → **human approves** → released to engineer.

7. Do & Don't for the orchestration layer

✓ Do	✗ Don't
Always pass a valid <code>asset_id</code> so RAG scopes correctly	Don't call RAG with a missing/guessed asset scope
Derive <code>degrading_sensors</code> • <code>fault_hypothesis</code> from ML output	Don't let an LLM invent which sensors moved or the RUL
Catch <code>NO_RELEVANT_PASSAGE</code> / <code>CORPUS_UNAVAILABLE</code> and route to human	Don't feed an empty/near-miss context to the generator
Keep a human gate before scheduling/shutdown	Don't let any agent commit an irreversible action alone

✓ Do	✗ Don't
Carry RUL + sensors + fault + citation into every work order	Don't output a work order the engineer can't trace back
Keep the supervisor + monitor on cheap models	Don't burn the strong model on routing/monitoring

8. Framework note

Designed framework-agnostic. Mapping:

- **LangGraph (recommended):** each agent = a node; shared state = the graph state object; abstention/confidence = conditional edges; human gates = interrupts. Best fit for the hierarchical routing + explicit state above.
- **CrewAI:** agents = crew members with roles; the shared state becomes task context passed between tasks; human gate = a human-input task.
- **AutoGen:** agents = conversable agents; supervisor = group-chat manager; tool calls = registered functions.



Bottom line: the layer is thin but critical — it reads ML signals, packages them into the RAG contract, guards against hallucination via abstention, and never lets an irreversible action skip a human. Build the state object + the ML → RAG translation first; the rest is wiring.

9. Open decisions (pick before coding)

1. **Framework** — you skipped this; LangGraph is the recommended default for the hierarchical + stateful design.
2. **Trigger mode** — real-time streaming per cycle vs. on-demand/batch scan of the fleet.
3. **Confidence-gate threshold** — the RAG similarity gate is fixed at 0.42; decide your *diagnosis* confidence cutoff for human routing.
4. **Where the fault-signature map lives** — in the orchestration layer (recommended) vs. inside RAG.

10. Build prompt (give this to your coding agent)

Paste this straight into your coding agent (Cursor / Claude Code / etc.). It encodes the exact interfaces from Shubham's ML pipeline and Ayush's RAG pipeline so the generated layer plugs in without rework.

```
You are building the AGENTIC ORCHESTRATION LAYER for "Mech Sage", a predictive-maintenance copilot for Ironside Manufacturing (fictional client). The ML pipeline and RAG pipeline are ALREADY BUILT and working. Do NOT rebuild them. Your job is ONLY the orchestration layer that sits on top of them.
```

```
FRAMEWORK: LangGraph (hierarchical, stateful graph). Python.
```

```
=== WHAT ALREADY EXISTS (do not modify, only call) ===
```

```
1) ML pipeline (NASA_CMAPSS_RUL_Project/src/)
```

- RUL prediction: LinearRegression on engineered features (rolling/expanding means, deltas, sliding windows; 129 features). Returns remaining useful life in cycles.
- Anomaly detection: IsolationForest for FD001/FD003 (unsupervised); LightGBM_Anomaly for FD002/FD004 (supervised, F1-tuned threshold). Returns anomaly flag + score.
- SHAP is available; top contributors include sensor_15 and sensor_7. Use SHAP contributions + recent sensor deltas to derive which sensors are degrading.

```
2) RAG pipeline (dev/rag/rag_pipeline.py)
```

- Entry tool: manual_retrieval_rag(asset_id, degrading_sensors, fault_hypothesis) [around L205]
- Pipeline is a thread-safe singleton via get_pipeline().
- asset_id SCOPES the corpus. Valid values: "turbofan" | "milling_machine" | "ironside".
- Two-stage retrieval: hybrid (dense HNSW + BM25) -> cross-encoder rerank -> parent-child lookup -> cosine validation.
- GUARDRAIL: if best similarity < 0.42 it returns an abstention error code: NO_RELEVANT_PASSAGE or CORPUS_UNAVAILABLE. Your layer MUST catch these and route to a human. NEVER feed an empty/near-miss context to the generator.
- Generation: Gemini under RAGAS faithfulness + answer-relevancy constraints.

```
=== THE LAYER YOU MUST BUILD ===
```

```
A 5-agent hierarchical LangGraph with a single shared state object.
```

```
SHARED STATE (typed; every node reads/writes its slice):
```

```
asset_id, raw_telemetry,  
rul_estimate, anomaly_flag, anomaly_score, degrading_sensors,  
fault_hypothesis, retrieved_passages,  
diagnosis, confidence, citation,  
work_order, schedule_proposal, approval_status
```

```
AGENTS (nodes):
```

1. Fleet Supervisor - cheap model, no tools. Triage the fleet, pick the asset, set ass

et_id (thus RAG scope). Routing only.

2. Per-Asset Monitor - ML BOUNDARY. Call RUL + anomaly models. Build degrading_sensors from SHAP top features + sensor deltas (with direction, e.g. ["sensor_15\u2191", "sensor_7\u2191"]). If not abnormal -> return to Supervisor, no RAG call.

3. Diagnostics - RAG BOUNDARY + strong model. Map degrading_sensors -> fault_hypothesis via a fault-signature lookup. Call manual_retrieval_rag(asset_id, degrading_sensors, fault_hypothesis). Catch NO_RELEVANT_PASSAGE / CORPUS_UNAVAILABLE -> set diagnosis="unresolved" and route to human. Owns the confidence gate: below cutoff -> human. On success, write diagnosis + confidence + citation.

4. Work-Order Drafting - mid model. Use returned parent passages (procedure + parts) to draft a structured work order in PLAIN PLANT LANGUAGE (asset id + location, priority, RUL, diagnosis, evidence, action, parts, est. downtime, safety/LOTO). Must stay grounded in retrieved passages only.

5. Scheduling - mid model, plan-and-execute. Propose a downtime slot, then STOP at a human-approval interrupt. Nothing irreversible commits without approval.

CONTROL FLOW:

Supervisor -> Monitor -> (normal: back to Supervisor | abnormal: Diagnostics)

Diagnostics -> (passages+confident: Work-Order | abstain/low-confidence: Human review)

Work-Order -> Scheduling -> Human approval interrupt -> release work order

HARD RULES:

- ML owns the numbers (RUL, anomaly). RAG owns the knowledge. LLM only reasons/phrases. Never let an LLM invent RUL, failure mode, or manual facts.

- degrading_sensors and fault_hypothesis are DERIVED FROM ML OUTPUT, never guessed by an LLM.

- Always pass a valid asset_id to the RAG tool.

- Abstention is a first-class path -> human, never a hallucinated answer.

- Human gate before any schedule/shutdown (LangGraph interrupt).

- Every work order must carry RUL + degrading_sensors + diagnosis + manual citation for traceability.

DELIVERABLES:

- state.py (typed shared-state / GraphState)

- agents/*.py (one node per agent)

- tools.py (thin wrappers: ml_predict(...) and manual_retrieval_rag(...) incl. abstention handling)

- graph.py (LangGraph wiring: nodes, conditional edges, human interrupts)

- a runnable demo on one C-MAPSS unit that produces a work order end-to-end and a second demo that triggers the abstain->human path.

Ask me for the exact ml_predict import path if it is not obvious from the repo. Keep the layer THIN: read ML signals, package the RAG contract, guard abstention, gate the human. Everything else is wiring.

After your agent generates this, run the two demos (happy path + abstain path) to confirm the wiring before integrating with the real repo imports.